NSCLC_Modeling_2

Setups

```
library(dplyr)
```

```
Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

    filter, lag

The following objects are masked from 'package:base':

    intersect, setdiff, setequal, union
```

```
library(survminer)
```

```
Loading required package: ggplot2

Warning: package 'ggplot2' was built under R version 4.4.3

Loading required package: ggpubr
```

```
library(LTRCforests)
```

```
Attaching package: 'LTRCforests'
```

```
The following object is masked from 'package:base':

    print
```

```r
library(ggplot2)
library(survival)
```

```
Attaching package: 'survival'

The following object is masked from 'package:survminer':

    myeloma
```

```r
nsclc <- read.csv("nsclc.csv") # read in the dataset

# Adjusting factor variables
# Adjusting factor variables
nsclc <- nsclc %>% mutate(
  stage_f = as.factor(stage_f),
  smoke_f = as.factor(smoke_f),
  prior_med_f = as.factor(prior_med_to_msk_f),
  gender = ifelse(GENDER == "Female", 1 ,0) # change Female to 1, and male to 0
) %>% select(-c(
  "GENDER", "STAGE_CDM_DERIVED",
  "SMOKING_PREDICTIONS_3_CLASSES", "PRIOR_MED_TO_MSK"
))
```

1. Cox - Rarity weight

```r
# 1. Define clinical columns to isolate the gene matrix
clinical_cols <- c(
  "X", "PATIENT_ID", "age_at_diagnosis",  "event",
  "entry_time", "exit_time", "smoke_f", "stage_f", "prior_med_to_msk_f",
  "prior_med_f", "gender"
)

# Isolate the true gene columns
gene_cols <- setdiff(colnames(nsclc), clinical_cols)
```

```r
gene_matrix <- nsclc %>%
  select(all_of(gene_cols)) %>%
  mutate(across(everything(), ~ as.numeric(as.character(.))))

# 2. Calculate N and df
N <- nrow(gene_matrix)
df <- colSums(gene_matrix, na.rm = TRUE)

# 3. Handle zero-variance genes
# If a gene has 0 mutations, log(N/0) equals Infinity. We must drop them.
# It is also highly recommended to drop "singletons" (genes mutated in only 1 or 2 patients)
# to prevent the Cox model from overfitting to a single patient's outcome.
min_mutations <- 3
genes_to_keep_tfidf <- names(df[df >= min_mutations])

# Subset matrix and frequencies to kept genes
gene_matrix_filtered <- gene_matrix[, genes_to_keep_tfidf]
df_filtered <- df[df >= min_mutations]

# 4. Calculate the IDF Weights
idf_weights <- log(N / df_filtered)

# 5. Apply weights (TF * IDF)
# sweep() multiplies each column in the matrix by its corresponding IDF weight
# If patient = 1 (mutated), they get 1 * IDF_weight. If 0, they get 0.
gene_matrix_weighted <- sweep(
  x = gene_matrix_filtered,
  MARGIN = 2,
  STATS = idf_weights,
  FUN = "*"
)

# 6. Recombine with clinical data
nsclc_tfidf <- bind_cols(nsclc[, clinical_cols], gene_matrix_weighted)

# Check your results
cat("Total patients (N):", N, "\n")
```

```
Total patients (N): 7802
```

```r
cat("Genes evaluated:", length(gene_cols), "\n")
```

```
Genes evaluated: 517
```

```r
cat("Genes kept for TF-IDF (>= 3 mutations):", length(genes_to_keep_tfidf), "\n")
```

```
Genes kept for TF-IDF (>= 3 mutations): 509
```

Data Preparation for glmnet

```r
library(glmnet)
```

```
Loading required package: Matrix
```

```
Loaded glmnet 4.1-10
```

```r
# 1. Isolate the predictor variables
x_vars <- nsclc_tfidf %>%
  select(-X, -PATIENT_ID, -entry_time, -exit_time, -event)

# 2. Create the predictor matrix
# The '~ . - 1' tells R to create dummy variables for your factors but omit the intercept
# (Cox models don't use a standard intercept in the x matrix)
x_matrix <- model.matrix(~ . - 1, data = x_vars)

# 3. Create the Survival response object
# glmnet accepts standard Surv() objects for the 'cox' family
y_surv <- Surv(
  time = nsclc_tfidf$entry_time,
  time2 = nsclc_tfidf$exit_time,
  event = nsclc_tfidf$event
)
```

data cleaning

```
library(tidyr)
```

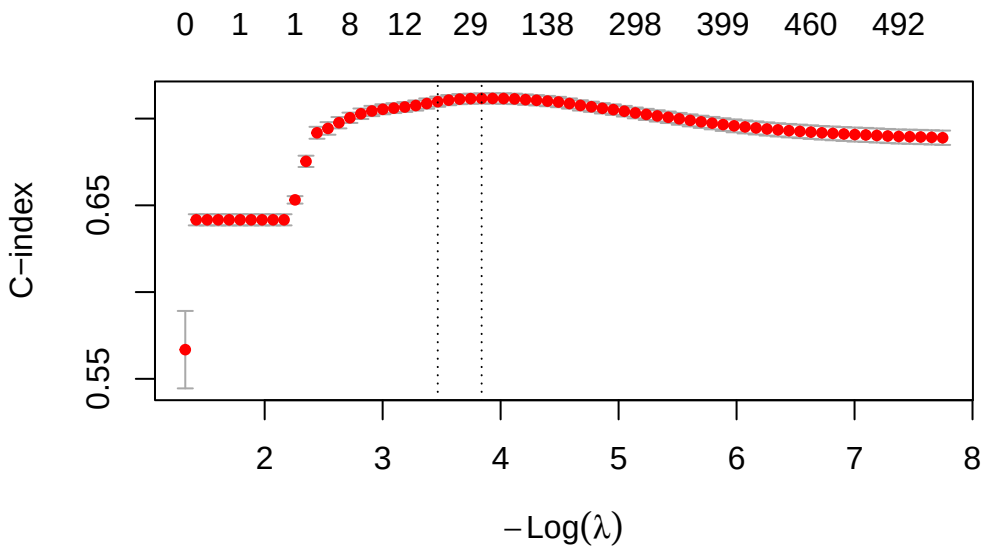
```
Attaching package: 'tidyr'

The following objects are masked from 'package:Matrix':

    expand, pack, unpack
```

```
# 1. Create a complete dataset with NO missing values
# This guarantees x and y will be perfectly aligned
nsclc_complete <- nsclc_tfidf %>% drop_na()

# 2. Isolate the predictor variables from the COMPLETE dataset
x_vars <- nsclc_complete %>%
  select(-X, -PATIENT_ID, -entry_time, -exit_time, -event)

# 3. Create the predictor matrix
x_matrix <- model.matrix(~ . - 1, data = x_vars)

# 4. Create the Survival response object from the SAME complete dataset
y_surv <- Surv(
  time = nsclc_complete$entry_time,
  time2 = nsclc_complete$exit_time,
  event = nsclc_complete$event
)
```

fit cross-validated lasso model

```
# Set seed for reproducibility in cross-validation
set.seed(629)

# Fit the Cross-Validated Lasso Cox Model
cv_lasso_cox <- cv.glmnet(
  x = x_matrix,
  y = y_surv,
  family = "cox",
  alpha = 1,          # alpha = 1 specifies LASSO (alpha = 0 would be Ridge)
  type.measure = "C"  # Optimize the penalty to maximize the Harrell's C-index
```

```
)

# Plot the cross-validation curve
# This shows the C-index on the y-axis against the log(lambda) penalty on the x-axis
plot(cv_lasso_cox)
```




```
# Print the performance metrics
cat("Optimal Lambda (maximizes C-index):", cv_lasso_cox$lambda.min, "\n")
```

```
Optimal Lambda (maximizes C-index): 0.02151293
```

```
cat("Max Cross-Validated C-index:", max(cv_lasso_cox$cvm, na.rm = TRUE), "\n")
```

```
Max Cross-Validated C-index: 0.7116163
```

Extract the "Winning" Features

```r
# Extract the coefficients at the optimal lambda value
lasso_coefs <- coef(cv_lasso_cox, s = "lambda.min")

# Filter for features that have a non-zero coefficient
active_features <- rownames(lasso_coefs)[which(lasso_coefs != 0)]

cat("Total features evaluated:", ncol(x_matrix), "\n")
```

```
Total features evaluated: 519
```

```r
cat("Features selected by LASSO:", length(active_features), "\n\n")
```

```
Features selected by LASSO: 36
```

```r
# View the final selected features and their weights
selected_weights <- lasso_coefs[active_features, , drop = FALSE]
print(selected_weights)
```

```
36 x 1 sparse Matrix of class "dgCMatrix"
                                                    1
age_at_diagnosis                           -4.697323e-03
smoke_fNever                               -6.835691e-02
smoke_fUnknown                              3.599768e-01
stage_fStage 4                              9.855162e-01
prior_med_to_msk_fPrior medications to MSK  2.335832e-01
prior_med_fPrior medications to MSK         1.812654e-15
gender                                     -2.100490e-01
TP53                                        3.782138e-01
PTPRD                                      -5.140251e-02
SMARCA4                                     6.551366e-02
PTPRT                                      -3.772974e-03
BLM                                        -1.627177e-02
MTOR                                       -4.700615e-03
KEAP1                                       1.396909e-01
ASXL1                                      -1.852914e-02
MED12                                      -5.737659e-05
EGFR                                       -1.344678e-01
CTCF                                       -3.144733e-03
STK11                                       1.145744e-01
EPHA3                                      -3.969681e-02
```

```
GRIN2A                                  6.166904e-03
NFE2L2                                  5.074541e-02
PAX5                                   -1.518080e-02
FGFR4                                  -1.093722e-02
RUNX1                                  -1.023719e-02
EP300                                   1.039709e-02
DNMT3A                                 -2.131559e-02
PIK3C3                                 -2.149143e-03
TGFBR2                                  6.506783e-02
PTEN                                    5.566474e-02
INPP4A                                  1.102064e-02
CDK12                                  -3.265524e-02
SPOP                                   -9.788005e-03
MSI1                                   -1.091910e-02
CSDE1                                  -5.361483e-03
MSI2                                   -3.462621e-02
```

Post-LASSO unpenalized Cox model.

```r
# 1. Identify ONLY the genes that LASSO selected
# We do this by cross-referencing the active features with your original gene list
selected_genes <- active_features[active_features %in% gene_cols]

# 2. Build a new dataframe using the ORIGINAL clinical columns + the winning genes
# This brings back your proper factor variables (stage_f, prior_med_f, etc.)
final_model_data_clean <- nsclc_complete %>%
  select(
    entry_time, exit_time, event,   # Survival outcomes
    age_at_diagnosis, smoke_f, stage_f, prior_med_f, gender, # Clinical vars
    all_of(selected_genes)          # Winning genes
  )

# 3. Run the final Cox model
# R will automatically handle the dummy coding behind the scenes
clean_cox_model <- coxph(
  Surv(entry_time, exit_time, event) ~ .,
  data = final_model_data_clean
)

# Check the beautiful, NA-free output
summary(clean_cox_model)
```

```
Call:
coxph(formula = Surv(entry_time, exit_time, event) ~ ., data = final_model_data_clean)

  n= 7470, number of events= 3838
```

	coef	exp(coef)	se(coef)	z
age_at_diagnosis	-0.009955	0.990095	0.001555	-6.400
smoke_fNever	-0.166073	0.846984	0.044133	-3.763
smoke_fUnknown	0.454420	1.575260	0.068520	6.632
stage_fStage 4	1.080267	2.945465	0.034829	31.016
prior_med_fPrior medications to MSK	0.364913	1.440389	0.044224	8.251
prior_med_fUnknown	0.166279	1.180902	0.054691	3.040
gender	-0.273098	0.761018	0.033230	-8.218
TP53	0.561711	1.753671	0.052515	10.696
PTPRD	-0.096354	0.908143	0.024994	-3.855
SMARCA4	0.125477	1.133689	0.021736	5.773
PTPRT	-0.061430	0.940419	0.024430	-2.515
BLM	-0.101576	0.903412	0.036964	-2.748
MTOR	-0.072434	0.930127	0.028957	-2.501
KEAP1	0.175499	1.191841	0.024015	7.308
ASXL1	-0.102950	0.902172	0.030459	-3.380
MED12	-0.053713	0.947704	0.026882	-1.998
EGFR	-0.203027	0.816256	0.030169	-6.730
CTCF	-0.075221	0.927538	0.036164	-2.080
STK11	0.163430	1.177543	0.024480	6.676
EPHA3	-0.097267	0.907314	0.025408	-3.828
GRIN2A	0.076851	1.079881	0.023524	3.267
NFE2L2	0.110013	1.116293	0.024568	4.478
PAX5	-0.088785	0.915043	0.038109	-2.330
FGFR4	-0.071722	0.930790	0.032850	-2.183
RUNX1	-0.093620	0.910629	0.038827	-2.411
EP300	0.080432	1.083755	0.024609	3.268
DNMT3A	-0.080811	0.922368	0.030838	-2.620
PIK3C3	-0.073169	0.929444	0.030385	-2.408
TGFBR2	0.121018	1.128646	0.026008	4.653
PTEN	0.107784	1.113808	0.023774	4.534
INPP4A	0.087886	1.091863	0.028875	3.044
CDK12	-0.111039	0.894904	0.033977	-3.268
SPOP	-0.117904	0.888782	0.045581	-2.587
MSI1	-0.130833	0.877364	0.055274	-2.367
CSDE1	-0.104512	0.900764	0.043490	-2.403
MSI2	-0.199125	0.819447	0.079682	-2.499
	Pr(>|z|)			

```
age_at_diagnosis                     1.55e-10 ***
smoke_fNever                         0.000168 ***
smoke_fUnknown                       3.31e-11 ***
stage_fStage 4                        < 2e-16 ***
prior_med_fPrior medications to MSK   < 2e-16 ***
prior_med_fUnknown                   0.002363 **
gender                                < 2e-16 ***
TP53                                  < 2e-16 ***
PTPRD                                0.000116 ***
SMARCA4                              7.80e-09 ***
PTPRT                                0.011917 *
BLM                                  0.005997 **
MTOR                                 0.012369 *
KEAP1                                2.72e-13 ***
ASXL1                                0.000725 ***
MED12                                0.045704 *
EGFR                                 1.70e-11 ***
CTCF                                 0.037527 *
STK11                                2.45e-11 ***
EPHA3                                0.000129 ***
GRIN2A                               0.001087 **
NFE2L2                               7.54e-06 ***
PAX5                                 0.019818 *
FGFR4                                0.029014 *
RUNX1                                0.015900 *
EP300                                0.001082 **
DNMT3A                               0.008781 **
PIK3C3                               0.016038 *
TGFBR2                               3.27e-06 ***
PTEN                                 5.80e-06 ***
INPP4A                               0.002337 **
CDK12                                0.001083 **
SPOP                                 0.009690 **
MSI1                                 0.017932 *
CSDE1                                0.016254 *
MSI2                                 0.012454 *
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

                                     exp(coef) exp(-coef) lower .95 upper .95
age_at_diagnosis                        0.9901     1.0100    0.9871    0.9931
smoke_fNever                            0.8470     1.1807    0.7768    0.9235
smoke_fUnknown                          1.5753     0.6348    1.3773    1.8017
```

stage_fStage 4	2.9455	0.3395	2.7511	3.1536
prior_med_fPrior medications to MSK	1.4404	0.6943	1.3208	1.5708
prior_med_fUnknown	1.1809	0.8468	1.0609	1.3145
gender	0.7610	1.3140	0.7130	0.8122
TP53	1.7537	0.5702	1.5821	1.9438
PTPRD	0.9081	1.1011	0.8647	0.9537
SMARCA4	1.1337	0.8821	1.0864	1.1830
PTPRT	0.9404	1.0634	0.8965	0.9865
BLM	0.9034	1.1069	0.8403	0.9713
MTOR	0.9301	1.0751	0.8788	0.9844
KEAP1	1.1918	0.8390	1.1370	1.2493
ASXL1	0.9022	1.1084	0.8499	0.9577
MED12	0.9477	1.0552	0.8991	0.9990
EGFR	0.8163	1.2251	0.7694	0.8660
CTCF	0.9275	1.0781	0.8641	0.9957
STK11	1.1775	0.8492	1.1224	1.2354
EPHA3	0.9073	1.1022	0.8632	0.9536
GRIN2A	1.0799	0.9260	1.0312	1.1308
NFE2L2	1.1163	0.8958	1.0638	1.1714
PAX5	0.9150	1.0928	0.8492	0.9860
FGFR4	0.9308	1.0744	0.8727	0.9927
RUNX1	0.9106	1.0981	0.8439	0.9826
EP300	1.0838	0.9227	1.0327	1.1373
DNMT3A	0.9224	1.0842	0.8683	0.9798
PIK3C3	0.9294	1.0759	0.8757	0.9865
TGFBR2	1.1286	0.8860	1.0726	1.1877
PTEN	1.1138	0.8978	1.0631	1.1669
INPP4A	1.0919	0.9159	1.0318	1.1554
CDK12	0.8949	1.1174	0.8373	0.9565
SPOP	0.8888	1.1251	0.8128	0.9718
MSI1	0.8774	1.1398	0.7873	0.9778
CSDE1	0.9008	1.1102	0.8272	0.9809
MSI2	0.8194	1.2203	0.7010	0.9580

```
Concordance= 0.721  (se = 0.004 )
Likelihood ratio test= 1987  on 36 df,   p=<2e-16
Wald test            = 1988  on 36 df,   p=<2e-16
Score (logrank) test = 2121  on 36 df,   p=<2e-16
```

c-index

```
# Setup CV
set.seed(629)
K <- 5
n <- nrow(final_model_data_clean)
fold_id <- sample(rep(1:K, length.out = n))
cv_cindex_post_lasso <- numeric(K)

for(k in 1:K){

  train_data <- final_model_data_clean[fold_id != k, ]
  test_data  <- final_model_data_clean[fold_id == k, ]

  # Fit the post-LASSO model on the training fold
  fit <- coxph(
    Surv(entry_time, exit_time, event) ~ .,
    data = train_data
  )

  # Predict risk scores on the test fold
  test_data$risk_score <- predict(fit, newdata = test_data, type = "lp", na.action = na.pass)

  # Compute C-index on the test fold
  c_obj <- concordance(
    Surv(entry_time, exit_time, event) ~ risk_score,
    data = test_data,
    reverse = TRUE
  )

  cv_cindex_post_lasso[k] <- c_obj$concordance
}

# Print results
cat("Post-LASSO Mean C-index:", mean(cv_cindex_post_lasso, na.rm = TRUE), "\n")
```

```
Post-LASSO Mean C-index: 0.7179219
```

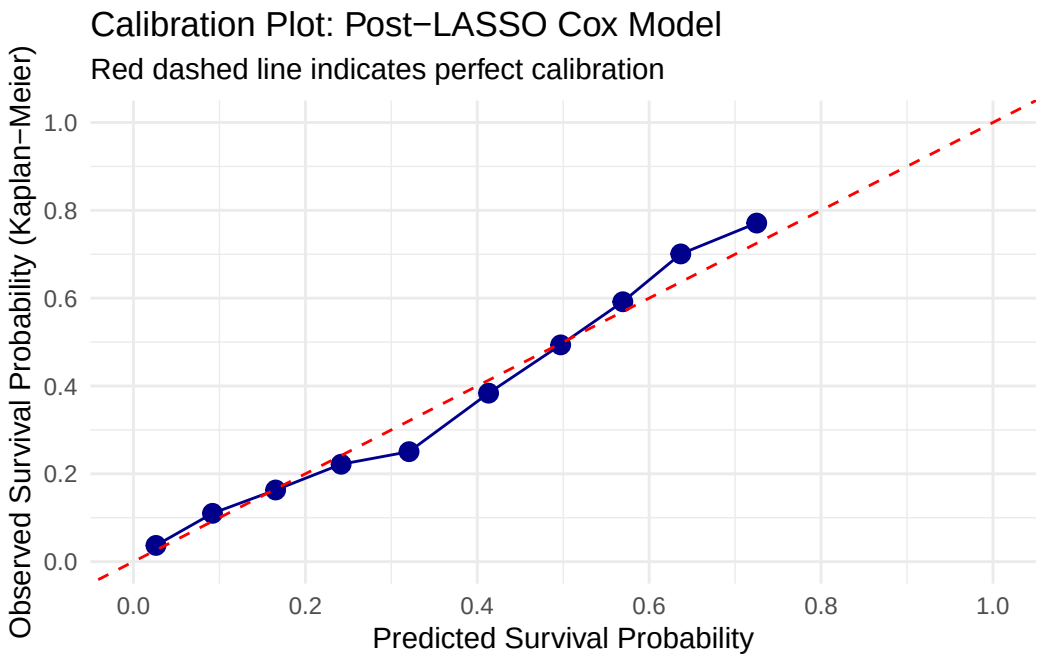
```
cat("Post-LASSO SD C-index:", sd(cv_cindex_post_lasso, na.rm = TRUE), "\n")
```

```
Post-LASSO SD C-index: 0.008623014
```

calibration plot:

```
library(ggplot2)

# Define the time horizon (e.g., 80th percentile of follow-up time)
time_horizon <- quantile(final_model_data_clean$exit_time, 0.80)

# Calculate predicted survival probabilities for the entire cohort at the time horizon
sf_final <- survfit(clean_cox_model, newdata = final_model_data_clean)
pred_surv <- summary(sf_final, times = time_horizon)$surv
pred_surv <- as.numeric(pred_surv)

# Add predictions to the dataset
final_model_data_clean$pred_surv <- pred_surv

# Create 10 risk groups (deciles) based on predicted survival
final_model_data_clean$decile <- ntile(final_model_data_clean$pred_surv, 10)

# Calculate observed Kaplan-Meier estimates for each decile
calibration_data <- final_model_data_clean %>%
  group_by(decile) %>%
  group_modify(~ {

    km_fit <- survfit(Surv(entry_time, exit_time, event) ~ 1, data = .x)

    km_summary <- summary(km_fit,
                          times = time_horizon,
                          extend = TRUE)

    data.frame(
      mean_pred = mean(.x$pred_surv, na.rm = TRUE),
      km_est = km_summary$surv
    )
  })

# Plot the calibration curve
ggplot(calibration_data, aes(x = mean_pred, y = km_est)) +
  geom_point(size = 3, color = "darkblue") +
  geom_line(color = "darkblue") +
  geom_abline(slope = 1, intercept = 0, linetype = "dashed", color = "red") +
  scale_x_continuous(limits = c(0, 1), breaks = seq(0, 1, 0.2)) +
  scale_y_continuous(limits = c(0, 1), breaks = seq(0, 1, 0.2)) +
```

```
  labs(
    x = "Predicted Survival Probability",
    y = "Observed Survival Probability (Kaplan-Meier)",
    title = "Calibration Plot: Post-LASSO Cox Model",
    subtitle = "Red dashed line indicates perfect calibration"
  ) +
  theme_minimal()
```



Time-Dependent ROC Curve

```
library(timeROC)

# 1. Generate the risk scores (linear predictors) from your final model
# We use the clean model and dataset from the previous step
final_model_data_clean$risk_score <- predict(clean_cox_model, type = "lp")

# 2. Define the time horizon (same as your calibration plot)
time_horizon <- quantile(final_model_data_clean$exit_time, 0.80)

# 3. Calculate the Time-dependent ROC
```

```r
roc_post_lasso <- timeROC(
  T = final_model_data_clean$exit_time,
  delta = final_model_data_clean$event,
  marker = final_model_data_clean$risk_score,
  cause = 1,
  times = time_horizon,
  iid = TRUE # Keeps the data needed to calculate Confidence Intervals
)

# 4. Extract the Area Under the Curve (AUC) and Confidence Intervals
auc_value <- roc_post_lasso$AUC[2] # Index 2 corresponds to the time_horizon we passed
se_value <- roc_post_lasso$inference$vect_sd_1[2]

# Calculate 95% CI
ci_lower <- auc_value - 1.96 * se_value
ci_upper <- auc_value + 1.96 * se_value

cat("Time Horizon:", round(time_horizon, 2), "\n")
```

```
Time Horizon: 66.4
```

```r
cat("AUC at Time Horizon:", round(auc_value, 3), "\n")
```

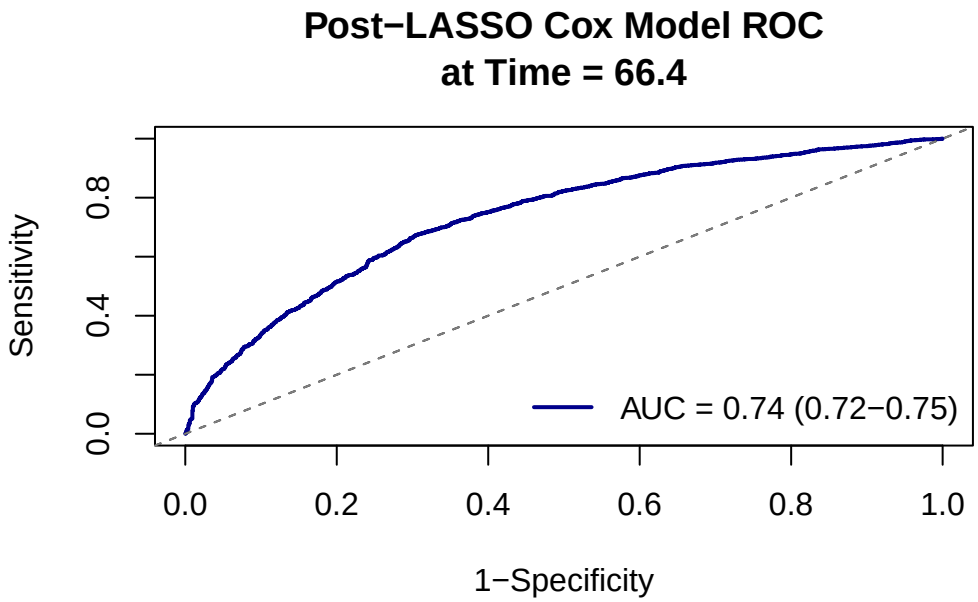
```
AUC at Time Horizon: 0.735
```

```r
cat("95% CI: [", round(ci_lower, 3), "-", round(ci_upper, 3), "]\n\n")
```

```
95% CI: [ 0.72 - 0.751 ]
```

```r
# 5. Plot the ROC Curve
{plot(roc_post_lasso,
     time = time_horizon,
     col = "darkblue",
     lwd = 2,
     title = FALSE)

# Add a reference line and a title
abline(0, 1, lty = 2, col = "gray50")
title(main = paste("Post-LASSO Cox Model ROC\nat Time =", round(time_horizon, 1)))
```

```
# Add a legend with the AUC value
legend("bottomright",
       legend = paste0("AUC = ", round(auc_value, 2),
                       " (", round(ci_lower, 2), "-", round(ci_upper, 2), ")"),
       col = "darkblue",
       lwd = 2,
       bty = "n")}
```



2. rsf

prepare the data by excluding the gene with frequency <= 1% (to reduce overfitting and increase interpretability)

```
library(dplyr)
library(LTRCtrees)
library(tidyr)

# 1. Define clinical columns to isolate the gene matrix
clinical_cols <- c(
  "X", "PATIENT_ID", "age_at_diagnosis",  "event",
```

```
  "entry_time", "exit_time", "smoke_f", "stage_f", "prior_med_to_msk_f",
  "prior_med_f", "gender"
)

# Isolate the true gene columns
# Train/Test Split (80/20) to prevent overfitting
gene_cols <- setdiff(colnames(nsclc), clinical_cols)
set.seed(629)
N_total <- nrow(nsclc)
train_idx <- sample(1:N_total, size = floor(0.8 * N_total))

train_raw <- nsclc[train_idx, ]
test_raw <- nsclc[-train_idx, ]

train_gene_matrix <- train_raw %>%
  select(all_of(gene_cols)) %>%
  mutate(across(everything(), ~ as.numeric(as.character(.))))

test_gene_matrix <- test_raw %>%
  select(all_of(gene_cols)) %>%
  mutate(across(everything(), ~ as.numeric(as.character(.))))

# Filter genes based strictly on TRAIN set frequencies (>= 1%)
N_train <- nrow(train_gene_matrix)
train_gene_frequencies <- colMeans(train_gene_matrix, na.rm = TRUE)
genes_to_keep_tfidf <- names(train_gene_frequencies[train_gene_frequencies >= 0.01])

# Subset both matrices to kept genes
train_gene_filtered <- train_gene_matrix[, genes_to_keep_tfidf]
test_gene_filtered <- test_gene_matrix[, genes_to_keep_tfidf]

# Calculate IDF Weights based ONLY on TRAIN set
df_train_filtered <- colSums(train_gene_filtered, na.rm = TRUE)
idf_weights <- log(N_train / df_train_filtered)

# Apply weights (TF * IDF) to TRAIN
train_gene_weighted <- sweep(train_gene_filtered, MARGIN = 2, STATS = idf_weights, FUN = "*")
train_complete <- bind_cols(train_raw[, clinical_cols], train_gene_weighted) %>% drop_na()

# Apply EXACT SAME weights to TEST (prevents data leakage)
test_gene_weighted <- sweep(test_gene_filtered, MARGIN = 2, STATS = idf_weights, FUN = "*")
test_complete <- bind_cols(test_raw[, clinical_cols], test_gene_weighted) %>% drop_na()
```

```r
cat("Train patients:", nrow(train_complete), "\n")
```

```
Train patients: 5979
```

```r
cat("Test patients:", nrow(test_complete), "\n")
```

```
Test patients: 1491
```

```r
cat("Genes kept for TF-IDF (>= 1% train frequency):", length(genes_to_keep_tfidf), "\n")
```

```
Genes kept for TF-IDF (>= 1% train frequency): 233
```

Fit the RSF on training Data

```r
clinical_features <- c("age_at_diagnosis", "smoke_f", "stage_f", "prior_med_f", "gender")
all_features <- c(clinical_features, genes_to_keep_tfidf)

rf_formula <- as.formula(
  paste("Surv(entry_time, exit_time, event) ~", paste(all_features, collapse = " + "))
)

m_try <- max(floor(sqrt(length(all_features))), 1)

set.seed(629)
rf_model_tfidf <- ltrcrrf(
  formula = rf_formula,
  data = train_complete,
  ntree = 500,
  nodesize = 20,
  mtry = m_try,
  nsplit = 10
)
```

Evaluation on Independent TEST Data

```r
time_horizon_tfidf <- quantile(train_complete$exit_time, 0.80)

# Predict survival probabilities on the TEST set
test_prob_obj <- predictProb(
  rf_model_tfidf,
  newdata = test_complete,
  time.eval = time_horizon_tfidf
)

test_complete$pred_surv <- as.numeric(test_prob_obj$survival.probs)
test_complete$risk_score <- 1 - test_complete$pred_surv

# --- A. C-index (Concordance) on TEST ---
c_obj <- concordance(
  Surv(entry_time, exit_time, event) ~ risk_score,
  data = test_complete,
  reverse = TRUE
)
cat("\nOut-of-Sample C-index:", round(c_obj$concordance, 3), "\n")
```

```
Out-of-Sample C-index: 0.689
```

Time-Dependent ROC on testing set

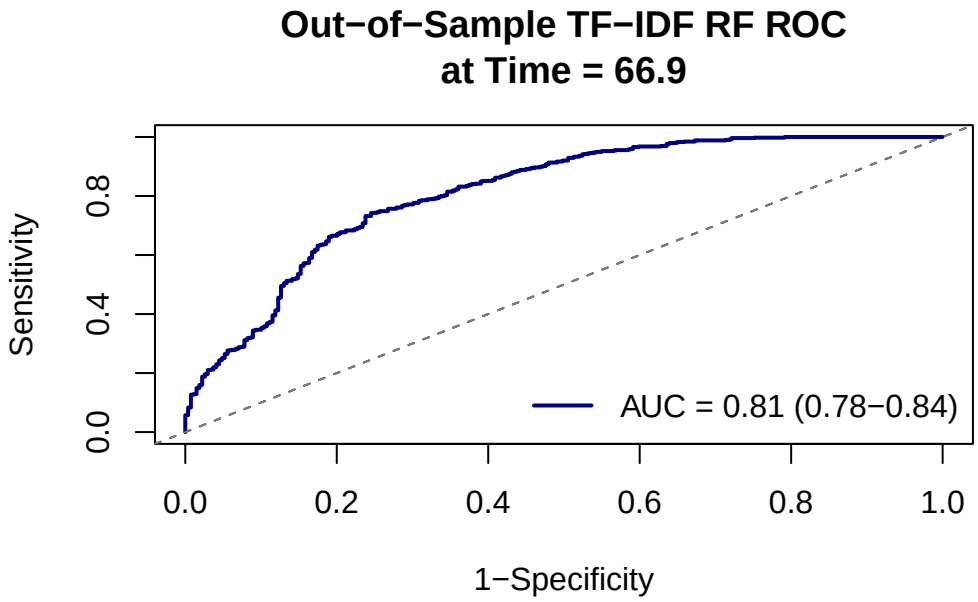
```r
roc_rf_tfidf <- timeROC(
  T = test_complete$exit_time,
  delta = test_complete$event,
  marker = test_complete$risk_score,
  cause = 1,
  times = time_horizon_tfidf,
  iid = TRUE
)

auc_val_rf_tfidf <- roc_rf_tfidf$AUC[2]
se_val_rf_tfidf <- roc_rf_tfidf$inference$vect_sd_1[2]
ci_lower_rf_tfidf <- auc_val_rf_tfidf - 1.96 * se_val_rf_tfidf
ci_upper_rf_tfidf <- auc_val_rf_tfidf + 1.96 * se_val_rf_tfidf

cat("Out-of-Sample AUC:", round(auc_val_rf_tfidf, 3), "\n")
```

```
Out-of-Sample AUC: 0.812
```

```
cat("95% CI: [", round(ci_lower_rf_tfidf, 3), "-", round(ci_upper_rf_tfidf, 3), "]\n\n")
```

```
95% CI: [ 0.78 - 0.844 ]
```

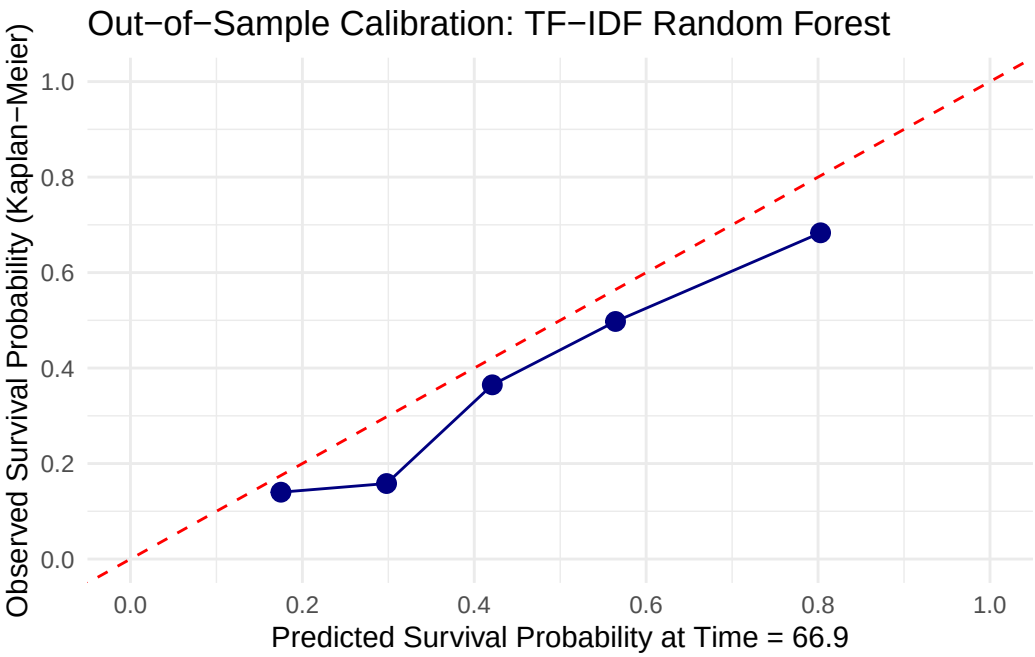
```
plot(roc_rf_tfidf, time = time_horizon_tfidf, col = "navy", lwd = 2, title = FALSE)
abline(0, 1, lty = 2, col = "gray50")
title(main = paste("Out-of-Sample TF-IDF RF ROC\nat Time =", round(time_horizon_tfidf, 1)))
legend("bottomright",
       legend = paste0("AUC = ", round(auc_val_rf_tfidf, 2),
                       " (", round(ci_lower_rf_tfidf, 2), "-", round(ci_upper_rf_tfidf, 2),
       col = "navy", lwd = 2, bty = "n")
```



Calibration Plot on testing set

```
# Note: Using 5 deciles (quintiles) instead of 10, as the test set is 20% of the total size.
# This prevents unstable Kaplan-Meier estimates caused by bins having too few events.
test_complete$decile <- ntile(test_complete$pred_surv, 5)
```

```r
calibration_data_rf_tfidf <- test_complete %>%
  group_by(decile) %>%
  group_modify(~ {
    km_fit <- survfit(Surv(entry_time, exit_time, event) ~ 1, data = .x)
    km_summary <- summary(km_fit, times = time_horizon_tfidf, extend = TRUE)
    data.frame(
      mean_pred = mean(.x$pred_surv, na.rm = TRUE),
      km_est = km_summary$surv
    )
  })

ggplot(calibration_data_rf_tfidf, aes(x = mean_pred, y = km_est)) +
  geom_point(size = 3, color = "navy") +
  geom_line(color = "navy") +
  geom_abline(slope = 1, intercept = 0, linetype = "dashed", color = "red") +
  scale_x_continuous(limits = c(0, 1), breaks = seq(0, 1, 0.2)) +
  scale_y_continuous(limits = c(0, 1), breaks = seq(0, 1, 0.2)) +
  labs(
    x = paste("Predicted Survival Probability at Time =", round(time_horizon_tfidf, 1)),
    y = "Observed Survival Probability (Kaplan-Meier)",
    title = "Out-of-Sample Calibration: TF-IDF Random Forest",
  ) +
  theme_minimal()
```


Out−of−Sample Calibration: TF−IDF Random Forest
Observed Survival Probability (Kaplan−Meier)
1.0
0.8
0.6
0.4
0.2
0.0
0.0
0.2
0.4
0.6
0.8
1.0
Predicted Survival Probability at Time = 66.9